

```
1
2 var gameConfig = {
3   'duration': 30000,
4   'presentation': false,
5   'ghost': false,
6   'ticker': false,
7   'gameid': false,
8   'stats': true,
9   'alwaysUseServer': false,
10  'autoplay': false,
11  'playable': true,
12  'logging': false,
13  'replay': false,
14  'saveServer': 'chartGame',
15  'readServer': 'chartGame',
16  'readServerLimit': 1000
17 }
18 // in cima a main.js, dopo le altre variabili
19 let titoliRend = []; // rendimento di ogni titolo
20 let sp500Rend = [];
21 let playedStockNames = []; // Add this line // rendimento
... relativo all'S&P500
22 // all config vars are configurable by query string on localhost;
23 // otherwise only ticker, gameid, and replay
24 if(window.location.hostname === 'localhost') {
25   Object.keys(gameConfig).forEach(function(key) {
26     var q = getQueryVariable(key);
27     if(q === 'true') {
28       gameConfig[key] = true;
29     } else if(q === 'false') {
30       gameConfig[key] = false;
31     } else if(q) {
32       gameConfig[key] = q;
33     }
34   })
35 } else {
36   gameConfig.ticker = getQueryVariable('ticker');
37   gameConfig.gameid = getQueryVariable('gameid');
38   gameConfig.replay = getQueryVariable('replay') === 'true' ? true :
... false;
39 }
40 var isTerminal = false;
41 if(gameConfig.replay) {
42   gameConfig.duration = 15000;
43   gameConfig.presentation = true;
44   gameConfig.ghost = true;
45   gameConfig.playable = false;
46 }
47
48 d3.select("body").classed("presentation", gameConfig.presentation);
49
```

```
50 // set of levels available
51 var tickers = [
52     {
53         "ticker": "aapl",
54         "name": "APPLE"
55     },
56     {
57         "ticker": "dell",
58         "name": "DELL COMPUTER"
59     },
60     {
61         "ticker": "cat",
62         "name": "CATERPILLAR"
63     },
64     {
65         "ticker": "coke",
66         "name": "COCA-COLA"
67     },
68     {
69         "ticker": "brkb",
70         "name": "BERKSHIRE HATHAWAY"
71     },
72     {
73         "ticker": "ene",
74         "name": "ENRON"
75     }
76 ,
77
78 {
79     "ticker": "fed",
80     "name": "SPX (Fed liftoff)"
81 },
82 {
83     "ticker": "nflx",
84     "name": "NFLX"
85 },
86 /*,
87 {
88     "ticker": "tsla",
89     "name": "TSLA US Equity"
90 },
91 {
92     "ticker": "brkb",
93     "name": "BRK/B US Equity"
94 },
95 {
96     "ticker": "mcz",
97     "name": "MCZ US Equity"
98 },
99 {
100     "ticker": "pg",
```

```
101     "name": "PG US Equity"
102 },
103 {
104     "ticker": "ene",
105     "name": "Enron US Equity"
106 },
107 {
108     "ticker": "znga",
109     "name": "ZNGA US Equity"
110 },
111 {
112     "ticker": "ge",
113     "name": "GE US Equity"
114 },
115 {
116     "ticker": "coke",
117     "name": "COKE US Equity"
118 },
119 {
120     "ticker": "googl",
121     "name": "GOOGL US Equity"
122 },
123 {
124     "ticker": "cat",
125     "name": "CAT US Equity"
126 },
127
128 // second batch
129
130 {
131     "ticker": "glen",
132     "name": "GLEN LN Equity"
133 },
134 {
135     "ticker": "mcd",
136     "name": "MCD US Equity"
137 },
138 {
139     "ticker": "dd",
140     "name": "DuPont US Equity"
141 },
142 {
143     "ticker": "xom",
144     "name": "XOM US Equity"
145 },
146 {
147     "ticker": "1788",
148     "name": "1788 HK (Guotai Junan)",
149     "description": "On Nov. 23, Guotai Junan announced its CEO was
... missing and could not be reached.
150 },
```

```
151 {
152   "ticker": "cag",
153   "name": "CAG US Equity"
154 },
155 {
156   "ticker": "f",
157   "name": "F US Equity"
158 },
159 {
160   "ticker": "corn",
161   "name": "CORN US Equity"
162 },
163 {
164   "ticker": "gpro",
165   "name": "GPRO US Equity"
166 },
167 {
168   "ticker": "yhoo",
169   "name": "YH00 US Equity"
170 },
171 {
172   "ticker": "lulu",
173   "name": "LULU (Lululemon)"
174 },
175 {
176   "ticker": "hal",
177   "name": "HAL (Halliburton)"
178 },
179 {
180   "ticker": "swhc",
181   "name": "SWHC (Smith & Wesson)"
182 },
183 {
184   "ticker": "kbio",
185   "name": "KBIO (KaloBios)"
186 },
187 {
188   "ticker": "cmg",
189   "name": "CMG (Chipotle)"
190 },
191 {
192   "ticker": "bks",
193   "name": "BKS (Barnes & Noble)"
194 },
195 {
196   "ticker": "gs",
197   "name": "GS (Goldman Sachs)"
198 },
199 {
200   "ticker": "dis",
201   "name": "DIS (Disney)"
```

```
202     },
203
204     {
205         "ticker": "fed",
206         "name": "SPX (Fed liftoff)"
207     },
208
209     {
210         "ticker": "hsi",
211         "name": "HSI (Hang Seng Index)"
212     },
213     {
214         "ticker": "shcomp",
215         "name": "SHCOMP (Shanghai Composite)"
216     }
217 */
218 ];
219
220 // UNUSED
221 /*
222     {
223         "ticker": "ibm", // shitty
224         "name": "IBM US Equity"
225     },
226     {
227         "ticker": "jpm",
228         "name": "JPM US Equity"
229     },
230     {
231         "ticker": "rtn",
232         "name": "RTN US Equity",
233         "description": "The previous day, SpaceX's rocket blew up. Not
... that it really mattered to Raytheon."
234     },
235     {
236         "ticker": "grpn", // way too flat intraday
237         "name": "GRPN US Equity"
238     },
239     {
240         "ticker": "7974",
241         "name": "7974 JT Equity (Nintendo)",
242         "description":
... "https://twitter.com/DavidInglesTV/status/659549109410332672"
243     },
244 */
245
246 tickers = _.shuffle(tickers);
247
248 var cash = 500;
249 var cashLiquid = true;
250 var levelNumber = 0;
```

```
251
252 var userId = Math.random().toString().split(".")[1];
253 var userHistory = {};
254
255 // persist userId and cash and userHistory
256 if(typeof(Storage) !== "undefined") {
257     if(!localStorage.getItem("userId")) {
258         try {
259             localStorage.setItem("userId", userId);
260         } catch(e) {
261             console.error(e);
262         }
263     } else {
264         userId = localStorage.getItem("userId");
265     }
266
267     if(!localStorage.getItem("cash")) {
268         try {
269             localStorage.setItem("cash", cash);
270         } catch(e) {
271             console.error(e);
272         }
273     } else {
274         cash = localStorage.getItem("cash");
275     }
276
277     if(!localStorage.getItem("userHistory")) {
278         try {
279             localStorage.setItem("userHistory",
280 ... JSON.stringify(userHistory));
281         } catch(e) {
282             console.error(e);
283         }
284     } else {
285         userHistory = JSON.parse(localStorage.getItem("userHistory"));
286     }
287 window.onunload = window.onbeforeunload = function() {
288     localStorage.setItem("cash", cash);
289     localStorage.setItem("userHistory", JSON.stringify(userHistory));
290 };
291
292 var indexFund;
293
294 var analysisStats = [];
295 var cashRules = [];
296
297 var tips = [
298     "Premi e tieni premuto in qualsiasi punto sul grafico per
299 ... comprare.",
300     "Mantieni premuto per mantenere la posizione del titolo e rilascia
```

```
299... per vendere.",
300     "Il trascinamento non ha alcun effetto. L'unico controllo è quello
... del tempo."
301 ];
302
303 var taglines = [
304     "Buy Low Sell High",
305     "So You Think You Can Trade?",
306     "America's Next Top Trader",
307     "Portrait of the Chartist as a Young Man",
308     "Get Rich Quick!",
309     "Terminal Professional Service Lite",
310     "Crushing Stock Saga",
311     "Flappy Stock",
312     "Uncharted 5",
313     "li'l charts",
314     "The Big Long"
315 ];
316
317 // initCanvas();
318
319 // gets s&p, then calls init
320 getIndexFund();
321
322 function getIndexFund() {
323     d3.csv("spx.csv", function(error, data) {
324
325         // more minimal version of processData below...
326         var parseDate = d3.time.format("%x").parse;
327         data.forEach(function(d) {
328             d.date = parseDate(d.date);
329             d.price = +d.price;
330         });
331         data.sort(function(a,b) {
332             return a.date - b.date;
333         });
334         indexFund = {
335             "ticker": "spx",
336             "name": "SPX",
337             "values": data
338         }
339
340         // ready to go
341         init();
342     });
343 }
344
345 function init() {
346
347     var isTerminal = false;
348     // find ticker specified in hash, if any
```

```
349   var tickerIndex = tickers.map(function(d) { return d.ticker;
...   }).indexOf(gameConfig.ticker);
350
351   var buttonText = gameConfig.replay ? "Watch" : "Play";
352
353   // FED LIFTOFF HACK
354   if(gameConfig.ticker) buttonText += " " +
...   gameConfig.ticker.toUpperCase()
355   //if(gameConfig.ticker && tickerIndex > -1) buttonText += " " +
...   tickers[tickerIndex].name;
356
357   if(gameConfig.gameid) buttonText += " challenge";
358   window.dvzqueue = queue;
359   d3.select(".opener")
360     .select("button")
361     .text(buttonText)
362     .style("background-color", "#ADD8E6") // This line sets the
...   background color to light blue
363     .on("click", getNextLevel)
364     .on("touchstart", function(d) { d3.select(this).classed("hover",
...   true); })
365     .on("touchend", function(d) { d3.select(this).classed("hover",
...   false); });
366
367   d3.select(".opener h3").text(_.sample(taglines));
368
369   if(gameConfig.autoplay) getNextLevel();
370 }
371
372 function getNextLevel() {
373   levelNumber++;
374
375   // this function is only called by "next" buttons, so... loading
...   indicators!
376   d3.select(this).text("...");
377
378   d3.select('#full-header').classed('hidden', true);
379
380   // find ticker specified in hash, if any
381   var tickerIndex = tickers.map(function(d) { return d.ticker;
...   }).indexOf(gameConfig.ticker);
382
383   if(gameConfig.ticker && tickerIndex !== -1) {
384     // load level specified in hash
385     loadTicker(tickers.splice(tickerIndex,1)[0]);
386   }
387   //else if(!isTerminal && innerWidth <= 1060 && levelNumber % 6 == 3)
...   {
388     //   loadAd();
389     // }
390   else if(tickers.length) {
```

```
391     loadTicker(tickers.pop());
392   } else {
393     // no more levels
394     gameOver();
395   }
396 }
397
398 // load ad
399 function loadAd() {
400
401   var i = d3.select(".levels").selectAll(".ad-item").size();
402
403   var adWrapper = d3.select(".levels")
404     .append("div.ad-item")
405     .style("z-index", d3.selectAll(".levels > div").size());
406
407   adWrapper.append("div.toolbar")
408     .append("button.next")
409     .text("Next")
410     .on("click", getNextLevel)
411     .on("touchstart", function(d) { d3.select(this).classed("hover",
... true); });
412     .on("touchend", function(d) { d3.select(this).classed("hover",
... false); });
413
414   adWrapper.append("p").text("Advertisement");
415
416   var ad = adWrapper.append("div.ad");
417
418   // get intended size from data attribute
419   // e.g. <div class="bannerad-sized" data-size="728,90"></div>
420   // fall back to widest IAB size that fits container
421   var size = ad.node().dataset.size ?
... ad.node().dataset.size.split(",") : getBestAdSize(ad.node());
422
423
424   ad.node().style.display = "block";
425   var randValue = new String(Math.random()).substring(2,11);
426   var n = i + 1;
427   ad.node().innerHTML = new_leader + '&position=box' + n + '&ord=' +
... randValue + '></iframe>';
428
429 }
430
431
432
433
434 // load file
435 function loadTicker(ticker) {
436   var qu = window.dvzqueue();
437   qu.defer(d3.csv, ticker.ticker + ".csv");
```

```
438
439   if(gameConfig.ghost) {
440       qu.defer(getTrades, ticker.ticker, undefined);
441   } else if(ticker.ticker === gameConfig.ticker && gameConfig.gameid)
442   {
443       qu.defer(getTrades, ticker.ticker, gameConfig.gameid);
444   }
445   qu.awaitAll(function(err, data) {
446
447       var stock = processData(ticker, data[0]);
448
449       if(data.length > 1) {
450           stock.competitor = data[1];
451           stock.competitor.trades.forEach(function(d,i) {
452               d.final = d[1];
453           });
454           stock.competitor.gameid = gameConfig.ghost ? "ghost" :
455   +gameConfig.gameid;
456       }
457
458       /// uhhhhh "GARBAGE COLLECTION" : )
459       d3.selectAll(".levels .chart-item").remove();
460
461       d3.select(".levels")
462         .style("z-index", "2")
463         .append("div.chart-item")
464         .style("z-index", d3.selectAll(".levels > div").size())
465         .datum(stock)
466         .call(playLevel);
467   });
468 }
469
470 // process data
471 function processData(stock, data) {
472
473     // console.log(stock.ticker);
474
475     // normalize length
476     data = data.slice(0,500);
477
478     // most are mm/dd/yyyy...
479     var parseDate = d3.time.format("%x").parse;
480     // but, uh, some are mm/dd/yyyy hh:mm:ss (24-hr) :-/
481     if(data[0].date.split(" ").length == 2) {
482         parseDate = d3.time.format("%x %X").parse;
483     }
484
485     data.forEach(function(d) {
486         d.date = parseDate(d.date);
```

```
487     d.price = +d.price;
488 });
489
490 data.sort(function(a,b) {
491     return a.date - b.date;
492 });
493
494 data.forEach(getStdev, data);
495 // data.forEach(getStdev2, data);
496
497 var t0 = _.min(data, f('date'));
498 var t1 = _.max(data, f('date'));
499 var stockReturn = (t1.price - t0.price) / t0.price;
500
501 function formatDate(d) {
502     // Assicuriamoci di gestire valori null/undefined
503     if (!d) return '-';
504     var date = new Date(d);
505     if (isNaN(date)) return '-'; // data non valida → segnaposto
506     var m = date.getMonth() + 1;
507     var day = date.getDate();
508     var y = date.getFullYear();
509     // zero-padding manuale, senza String.padStart (compatibilità
510     ... vecchi browser)
511     m = (m < 10 ? '0' : '') + m;
512     day = (day < 10 ? '0' : '') + day;
513     return m + '-' + day + '-' + y;
514 }
515
516 if(indexFund) {
517     var spxt0 = closest(f('date'))(indexFund.values, t0.date);
518     var spxt1 = closest(f('date'))(indexFund.values, t1.date);
519     var indexReturn = (spxt1.price - spxt0.price) / spxt0.price;
520 }
521
522 return {
523     "ticker": stock.ticker,
524     "name": stock.name,
525     "values": data,
526     "getSmoothDomain": getSmoothDomainFunction(data),
527     "period": Math.min(data[1].date - data[0].date, data[2].date -
528     ... data[1].date),
529     "trades": [],
530     "cash": [],
531     "stockReturn": stockReturn,
532     "indexReturn": indexReturn ? indexReturn : stockReturn,
533     "traderReturn": null,
534     "sharesHeld": 0,
535     "challengeUrl": window.location.href,
536     "spxStart": spxt0.price,
```

```
536     "spxEnd":    spxt1.price,
537     "spxStartDate": spxt0.date,
538     "spxEndDate":    spxt1.date
539   };
540 }
541
542 // render
543 function playLevel(selection) {
544   selection.each(function(stock) {
545
546     var item = d3.select(this),
547         margin = {top: 50, right: 40, bottom: 20, left: 0},
548         width = this.offsetWidth - margin.left - margin.right,
549         height = this.offsetHeight - margin.top - margin.bottom,
550         duration = gameConfig.duration,
551         timeframe = stock.period * (width / 8),
552         timeBounds,
553         priceBounds,
554         latest;
555
556     stock.cash.push({
557       "date": d3.min(stock.values, f('date')),
558       "cash": cash
559     });
560
561     // DEBUG
562     // if(gameConfig.presentation) {
563     //   margin.top = 5;
564     //   height = this.offsetHeight - margin.top - margin.bottom;
565     // }
566     // console.log(stock);
567     // console.log(stock.ticker, stock.values.length, stock.period /
568     ... (24 * 60 * 60 * 1000))
569     // console.log(d3.mean(stock.values.map(f('σ'))));
570     // console.log(stock.period / (24 * 60 * 60 * 1000));
571     // END DEBUG
572
573     var showTipThrottled = _.throttle(showTip, 5000, {leading:
574     ... false});
575
576     var dollarFormat = d3.format('$.2f');
577     var dollarFormat0 = d3.format('$.0f');
578     var percentFormat = d3.format(".1%");
579     var percentFormat0 = d3.format(".0%");
580
581     var xTickScale = d3.scale.linear()
582       .domain([320, 1280])
583       .range([4, 10]);
584
585     var colorScale = d3.scale.linear()
586       .domain([-0.01, 0.01])
```

```
585     .range(["#ff0000", "#00ff00"])
586     .clamp(true);
587
588     var iconNumberScale = d3.scale.log()
589       .domain([.1, 1000000])
590       .range([1, 10]);
591
592     var x = d3.time.scale()
593       .domain([
594         d3.min(stock.values, f('date')),
595         d3.max(stock.values, f('date'))
596       ])
597       .range([0, width]);
598
599     var y = d3.scale.linear()
600       .domain(d3.extent(stock.values, f('price')))
601       .range([height, 0]);
602
603     var tScale = d3.time.scale()
604       .domain(d3.extent(stock.values, f('date')))
605       .range([0, duration]);
606
607     var xAxis = d3.svg.axis()
608       .scale(x)
609       .ticks(xTickScale(width))
610       .orient("bottom");
611
612     var yAxis = d3.svg.axis()
613       .scale(y)
614       .orient("right")
615       .tickFormat(d3.format("$"));
616
617     var gridlines = d3.svg.axis()
618       .scale(y)
619       .tickSize(-width, 0)
620       .tickValues(gridlineTicks)
621       .orient("left");
622
623     var stepLine = d3.svg.line()
624       .x(function(d) { return x(d.date); })
625       .y(function(d) { return y(d.price); })
626       .interpolate("step-after");
627
628     var stepArea = d3.svg.area()
629       .x(function(d) { return x(d.date); })
630       .y0(y.range()[0])
631       .y1(function(d) { return y(d.price); })
632       .interpolate("step-after");
633
634     var line = d3.svg.line()
635       .x(function(d) { return x(d.date); })
```

```
636         .y(function(d) { return y(d.price); });
637
638     // DOM-building begins here
639
640     var toolbar = item.append("div.toolbar");
641     var alerts = item.append("div.alerts");
642
643     var timeLabel =
... toolbar.append("div.time").text(timerFormat(duration));
644     var cashHolder = toolbar.append("div.cash");
645     var cashLabel =
... cashHolder.append("span.net").text(dollarFormat(cash));
646     var cashChange = cashHolder.append("div.change");
647     var tickerLabel =
... toolbar.append("div.ticker").text(stock.name).style("display",
... "none");
648     if(gameConfig.replay) tickerLabel.style("display", "");
649
650     var svg = item.append("svg")
651         .attr("width", width + margin.left + margin.right)
652         .attr("height", height + margin.top + margin.bottom)
653         .append("g")
654         .attr("transform", "translate(" + margin.left + "," +
... margin.top + ")");
655
656     var clipPath = svg.append("clipPath")
657         .attr("id", "plot-area-clip")
658         .append("rect")
659         .attr("x", 0)
660         .attr("y", 0)
661         .attr("width", width)
662         .attr("height", height);
663
664     var gridlinesG = svg.append("g")
665         .attr("class", "gridlines axis")
666         .call(gridlines);
667
668     var xAxisG = svg.append("g")
669         .attr("class", "x axis")
670         .attr("transform", "translate(0," + height + ")")
671         .call(xAxis);
672
673     var yAxisG = svg.append("g")
674         .attr("class", "y axis")
675         .attr("transform", "translate(" + width + "," + 0 + ")")
676         .call(yAxis);
677
678     var lineArea = svg.append("path")
679         .datum(stock.values)
680         .attr("class", "area")
681         .attr("clip-path", "url(#plot-area-clip)");
```

```
682         .attr("d", stepArea);
683
684     var linePath = svg.append("path")
685         .datum(stock.values)
686         .attr("class", "line share")
687         .attr("clip-path", "url(#plot-area-clip)")
688         .attr("d", stepLine);
689
690     var yMark = svg.append("g.y-mark")
691         .attr("transform", "translate(" + (width + 3) + "," + 0 +
...   ")");
692     yMark.append("path")
693         .attr("d", d3.svg.symbol().type("triangle-up").size(30))
694         .style("transform", "rotate(-90deg)")
695         .style("fill", "#fff");
696     yMark.append("image")
697         .attr("xlink:href", "certificate-white.png")
698         .attr("width", 18)
699         .attr("height", 12)
700         .attr("x", 6)
701         .attr("y", -6);
702
703     svg.append("circle.cursor-indicator")
704         .attr("cx", "50")
705         .attr("cy", "50")
706         .attr("r", 1e-6);
707
708     tick(0);
709     startLevel();
710
711     //////////////////////////////////////
712
713     function startLevel() {
714
715         item.classed("playing", true);
716         d3.select("body").classed("playing", true);
717
718         if(gameConfig.playable) {
719             var dragger = d3.behavior.drag()
720                 .on("dragstart", dragstart)
721                 .on("drag", drag)
722                 .on("dragend", dragend);
723
724             item
725                 .call(dragger)
726                 .on("mousemove.cursor", mousemove);
727         }
728
729         d3.timer(tick);
730     }
731
```

```
732     function tick(t) {
733
734         // level is over
735         if(t > tScale.range()[1]) {
736             // liquidate any remaining shares
737             dragend();
738             // unbind trade listeners
739             item.on(".drag", null);
740             item.on(".cursor", null);
741         }
742
743         // heyo adaptive feedback!
744         if(!(d3.max(stock.trades, function(d) { return tScale(d[1].date)
... - tScale(d[0].date); }) > 500)) {
745             showTipThrottled();
746         }
747
748         timeBounds = [new Date(+tScale.invert(t) - timeframe),
... tScale.invert(t)];
749         priceBounds = stock.getSmoothDomain(tScale.invert(t));
750
751         x.domain(timeBounds);
752         y.domain(priceBounds);
753
754         xAxisG.call(xAxis);
755         yAxisG.call(yAxis);
756         gridlinesG.call(gridlines);
757         linePath.attr("d", stepLine);
758         lineArea.attr("d", stepArea);
759
760         var boundedData = stock.values.filter(function(d) { return
... d.date > timeBounds[0] && d.date <= timeBounds[1]; });
761         if(boundedData.length) {
762             latest = _.max(boundedData, function(d) { return d.date; });
763         }
764         if(!cashLiquid) {
765             cash = latest.price * stock.sharesHeld;
766         }
767
768         stock.trades.filter(function(d) { return d.outstanding;
... }).forEach(function(d) {
769             d[1] = {
770                 "date": timeBounds[1],
771                 "price": latest.price
772             };
773         });
774
775         yMark.attr("transform", "translate(" + (width + 3) + "," +
... y(latest.price) + ")");
776
777         svg.selectAll("path.transaction.live")
```

```
778     .data(stock.trades)
779     .enter()
780     .append("path.transaction.live");
781
782     svg.selectAll("path.transaction.live")
783     .attr("d", line)
784     .classed("outstanding", f('outstanding'))
785     .style("stroke", function(d) { return colorScale( d[1].price -
... d[0].price )});
786
787     cashLabel.text(dollarFormat(cash))
788     .classed("liquid", cashLiquid);
789     timeLabel.text(timerFormat(duration-t));
790
791     if(stock.competitor) {
792
793         var ghostTrades = stock.competitor.trades
794             .slice(0,1500)
795             .filter(function(d,i) {
796                 return d[0].date < timeBounds[1];
797             });
798         ghostTrades.forEach(function(d,i) {
799             if (d.final.date < timeBounds[1]) {
800                 // use final data
801                 d[1] = d.final;
802             } else {
803                 // trade is outstanding;
804                 // use then-latest data
805                 d[1] = {
806                     "date": timeBounds[1],
807                     "price": latest.price
808                 };
809             }
810         });
811
812         svg.selectAll("path.transaction.competitor")
813             .data(ghostTrades)
814             .enter()
815             .append("path.transaction.competitor")
816             .classed("ghost", stock.competitor.trades.length > 100);
817
818         svg.selectAll("path.transaction.competitor")
819             .attr("d", line)
820             .style("stroke", function(d) { return colorScale(d[1].price
... - d[0].price); });
821     }
822
823     // level is over, pt. 2 :-/
824     if(t > tScale.range()[1]) {
825         return levelOver();
826     };
```

```
827
828     }
829
830     function dragstart() {
831
832         // stock is already being traded.
833         // sloppy hack away from per-share trading...
834         if(!cashLiquid) return;
835         cashLiquid = false;
836
837         item.classed("holding", true);
838
839         var point = d3.mouse(svg.node());
840
841         stock.sharesHeld = cash / latest.price;
842
843         var trade = [
844             {
845                 "date": timeBounds[1],
846                 "price": latest.price
847             },
848             {
849                 "date": timeBounds[1],
850                 "price": latest.price
851             }
852         ];
853         trade.outstanding = true;
854         trade.id = Math.random().toString().split(".")[1];
855
856         stock.trades.push(trade);
857     /*
858         svg.selectAll("text.dollabill.id"+trade.id)
859             .data(d3.range(Math.round(iconNumberScale(trade[0].price *
860 ... stock.sharesHeld))))
861             .enter()
862             .append("text.dollabill.id"+trade.id)
863             .text("$")
864             .attr("x", function(d,i) { return Math.random() * 30 - 15; })
865             .attr("y", function(d,i) { return Math.random() * 30 - 15; })
866             .transition()
867             .duration(500)
868             .delay(function(d,i) { return 10*i; })
869             .ease("linear")
870             .attr("x", x(trade[0].date))
871             .attr("y", y(trade[0].price))
872             .remove();
873         svg.selectAll("image.shareicon.id"+trade.id)
874             .data(d3.range(Math.round(iconNumberScale(stock.sharesHeld))))
875             .enter()
876             .append("image.shareicon.id"+trade.id)
877             .attr("xlink:href","certificate-white.png")
```

```
877     .attr("width", 18)
878     .attr("height", 12)
879     .attr("x", function(d,i) { return x(trade[0].date) +
... (Math.random() * 30 - 15); })
880     .attr("y", function(d,i) { return y(trade[0].price) +
... (Math.random() * 30 - 15); })
881     .transition()
882     .duration(500)
883     .delay(function(d,i) { return 20*i; })
884     .ease("linear")
885     .attr("x", point[0])
886     .attr("y", point[1])
887     .remove();
888 */
889     svg.select(".cursor-indicator")
890     .attr("cx", point[0])
891     .attr("cy", point[1])
892     .transition()
893     .duration(250)
894     .attr("r", "25");
895
896 }
897
898 function drag() {
899
900     var point = d3.mouse(svg.node());
901
902     svg.select(".cursor-indicator")
903     .attr("cx", point[0])
904     .attr("cy", point[1]);
905 }
906
907 function mousemove() {
908     if(!cashLiquid) return;
909     svg.select(".cursor-indicator")
910     .attr("cx", d3.mouse(svg.node())[0])
911     .attr("cy", d3.mouse(svg.node())[1])
912     .transition()
913     .duration(250)
914     .attr("r", 5);
915 }
916
917 function dragend() {
918
919     item.classed("holding", false);
920
921     // liquidate any outstanding transactions
922     var outstandingTransactions = stock.trades.filter(function(d) {
923         return d.outstanding;
924     });
925     outstandingTransactions.forEach(function(d) {
```

```
926         d.outstanding = false;
927
928         cash = d[1].price * stock.sharesHeld;
929
930         // terrible hacky thing for zero-duration trades
931         // cf.
932         https://github.com/bizweekgraphics/stock-chart-game/issues/31
933         if(d[1].date - d[0].date) {
934             cashChangeValue = (d[1].price - d[0].price) *
935             stock.sharesHeld;
936             cashChange.text(dollarFormat(cashChangeValue))
937                 .style("color", colorScale(cashChangeValue))
938                 .style("opacity", 1)
939                 .transition()
940                 .duration(1000)
941                 .style("opacity", 0);
942         }
943     /*
944         svg.selectAll("text.dollabill.id"+d.id)
945             .data(d3.range(Math.round(iconNumberScale(d[0].price *
946             stock.sharesHeld))))
947             .enter()
948             .append("text.dollabill.id"+d.id)
949             .text("$")
950             .attr("x", x(d[1].date))
951             .attr("y", y(d[1].price));
952         svg.selectAll("text.dollabill.id"+d.id).transition()
953             .duration(500)
954             .delay(function(d,i) { return 10*i; })
955             .ease("linear")
956             .attr("x", function(d,i) { return Math.random() * 30 - 15;
957             ... })
958             .attr("y", function(d,i) { return Math.random() * 30 - 15;
959             ... })
960             .remove();
961         svg.selectAll("image.shareicon.id"+d.id)
962             .data(d3.range(Math.round(iconNumberScale(stock.sharesHeld))))
963             .enter()
964             .append("image.shareicon.id"+d.id)
965             .attr("xlink:href", "certificate-white.png")
966             .attr("width", 18)
967             .attr("height", 12)
968             .attr("x", d3.select(".cursor-indicator").attr("cx"))
969             .attr("y", d3.select(".cursor-indicator").attr("cy"));
970         svg.selectAll("image.shareicon.id"+d.id).transition()
971             .duration(500)
972             .delay(function(d,i) { return 20*i; })
973             .ease("linear")
974             .attr("x", function() { return x(d[1].date) + (Math.random()
975             ... * 30 - 15); })
```

```
970         .attr("y", function() { return y(d[1].price) +
... (Math.random() * 30 - 15); })
971         .remove();
972 */
973     svg.select(".cursor-indicator")
974         .transition()
975         .duration(250)
976         .attr("r", 1e-6);
977
978     stock.sharesHeld = 0;
979     cashLiquid = true;
980
981 });
982
983 }
984
985 function levelOver() {
986     item.classed("playing", false);
987     d3.select("body").classed("playing", false);
988     item.classed("level-over", true);
989
990     d3.select('meta[property="og:longTitle"]').attr("content", "I've
... made " + dollarFormat0(cash)
991         + " playing the @business Trading Game. Can you beat the
... market? Play now:");
992
993     tickerLabel.style("display", "");
994
995     svg.select(".cursor-indicator")
996         .transition()
997         .attr("r", 1e-6);
998
999     x.domain(d3.extent(stock.values, f('date')));
1000
1001     // fed liftoff hack
1002     if( d3.max(stock.values, f('price')) > 1000 ) {
1003         y.domain(d3.extent(stock.values, f('price')));
1004     } else {
1005         y.domain([0, d3.max(stock.values, f('price'))]);
1006     }
1007
1008     var transition = svg.transition()
1009         .delay(250)
1010         .duration(1000);
1011
1012     transition.select(".x.axis").call(xAxis);
1013     transition.select(".y.axis").call(yAxis);
1014     transition.select(".gridlines.axis").call(gridlines);
1015
1016     transition.select("path.line.share")
1017         .attr("d", function(d) { return stepLine(stock.values); });
```

```
1018     transition.select("path.area")
1019         .attr("d", function(d) { return stepArea(stock.values); });
1020
1021     transition.selectAll("path.transaction")
1022         .attr("d", line);
1023
1024     transition.select(".y-mark")
1025         .attr("transform", "translate(" + (width + 3) + "," +
... y(latest.price) + ")")
1026         .style("opacity", 0);
1027
1028     // button to advance
1029     toolbar.append("button.next")
1030         .text("Next")
1031         .on("click", getNextLevel)
1032         .on("touchstart", function(d) {
... d3.select(this).classed("hover", true); })
1033         .on("touchend", function(d) { d3.select(this).classed("hover",
... false); });
1034
1035     // update stock properties
1036     stock.cash.push({
1037         "date": d3.max(stock.values, f('date')),
1038         "cash": cash
1039     });
1040     stock.traderReturn = (stock.cash[1].cash - stock.cash[0].cash) /
... stock.cash[0].cash;
1041
1042     if(userHistory[stock.ticker] && stock.traderReturn >
... userHistory[stock.ticker].highscore) {
1043         addAlert("Record personale !", true);
1044     }
1045
1046     if(stock.competitor && stock.competitor.returns.length === 1) {
1047         if(stock.traderReturn > stock.competitor.returns[0]) {
1048             addAlert("You beat your challenger " +
... percentFormat0(stock.traderReturn) + " to " +
... percentFormat0(stock.competitor.returns[0]) + "!", true);
1049         } else {
1050             addAlert("You lost to your challenger, " +
... percentFormat0(stock.traderReturn) + " to " +
... percentFormat0(stock.competitor.returns[0]) + ".", true);
1051         }
1052     }
1053
1054     // update user history
1055     if(userHistory[stock.ticker]) {
1056         userHistory[stock.ticker].attempts++;
1057         userHistory[stock.ticker].highscore =
... Math.max(userHistory[stock.ticker].highscore, stock.traderReturn)
```

```
1059     } else {
1060         userHistory[stock.ticker] = {
1061             "attempts": 1,
1062             "highscore": stock.traderReturn
1063         }
1064     }
1065     // - adesso - (numeri puri)
1066     titoliRend.push(stock.traderReturn); // 0.352
1067     sp500Rend.push(stock.indexReturn); // -0.077
1068     playedStockNames.push(stock.name); // Add this line
1069     var alertText =
1070         '<p><strong>Il titolo era ' + stock.name + ' tra ' +
1071         stock.spxStartDate + ' e ' + stock.spxEndDate + '</strong></p>' +
1072         '<p>La tua strategia ha reso il <strong>' +
1073         percentFormat(stock.traderReturn) + '</strong></p>' +
1074         '<p>La strategia BUY&HOLD del titolo ha reso il <strong>' +
1075         percentFormat(stock.stockReturn) + '</strong></p>' +
1076         '<p>La strategia BUY&HOLD dell\'S&P&nbsp;500 ha reso il
1077         <strong>' + percentFormat(stock.indexReturn) + '</strong></p>' +
1078         '<p>S&P&nbsp;500 iniziale (' + stock.spxStartDate + '):
1079         <strong>' + stock.spxStart.toFixed(2) + '</strong> - finale (' +
1080         stock.spxEndDate + '): <strong>' + stock.spxEnd.toFixed(2) +
1081         '</strong></p>';
1082     ;
1083
1084     addAlert(alertText, true);
1085
1086     // Commenta questo blocco per disabilitare il salvataggio
1087     /*
1088     if(gameConfig.playable) {
1089         saveTrades(stock, function(err, key) {
1090
1091             stock.challengeUrl = window.location.protocol
1092                 + "://"
1093                 + window.location.host
1094                 + window.location.pathname
1095                 + '?ticker='+key.ticker + '&gameid=' + key.timestamp;
1096
1097             // history.pushState(null, null, stock.challengeUrl);
1098
1099             var alertText = 'Challenge someone to beat your score with
1100             this URL: <input value="' + stock.challengeUrl + '">';
1101             addAlert(alertText, true);
1102         });
1103     }
1104     */
1105     getTrades(stock.ticker, undefined, renderTrades);
1106
1107     function renderTrades(err, result) {
```

```
1102     if(err) {
1103         console.log(err);
1104         return false;
1105     }
1106
1107     if(window.innerWidth > 800 && !gameConfig.ghost) {
1108         svg.selectAll("path.transaction.past")
1109             .data(result.trades)
1110             .enter()
1111             .append("path.transaction.past")
1112             .attr("d", line)
1113             .style("stroke", function(d) { return
... colorScale(d[1].price - d[0].price); })
1114             .style("opacity", 0)
1115             .transition()
1116             .duration(1000)
1117             .delay(1000)
1118             .style("opacity", .05);
1119     }
1120
1121     result.returns.sort(function(a,b){return a - b});
1122     var percentile = result.returns.length ?
... (d3.bisect(result.returns, stock.traderReturn) /
... result.returns.length) : 1;
1123     var rank = result.returns.length - d3.bisect(result.returns,
... stock.traderReturn) + 1;
1124
1125     var performanceComments = [
1126         [0, "Being so bad takes skill."],
1127         [.25, "I suck!"],
1128         [.4, "That's, like, almost average?"],
1129         [.45, "Perfectly average is a kind of perfect."],
1130         [.55, "Above average!"],
1131         [.8, "I should do this for real!"],
1132         [.9, "Wicked hella good!"],
1133         [.99, "Holy shit!!"]
1134     ];
1135     var performanceCommentsScale = d3.scale.threshold()
1136         .domain(performanceComments.map(f(0)).slice(1))
1137         .range(performanceComments.map(f(1)));
1138
1139     var alertText = "You outperformed " +
... percentFormat(percentile)
1140         + " of players, ranking #" + rank + ". <span
... class='tweet'>Tweet</span>";
1141     var tweetText = "I beat " + percentFormat0(percentile)
1142         + " of people trading $" + stock.ticker.toUpperCase()
1143         + ", ranking #" + rank + ". "
1144         + performanceCommentsScale(percentile) + " The Trading
... Game:";
1145     addAlert(alertText, true, tweetText);
```

```
1146
1147     if(gameConfig.stats) {
1148
1149         var returnsAnnotations = [
1150             {"x": stock.stockReturn, "text":
... stock.ticker.toUpperCase()},
1151             {"x": stock.indexReturn, "text": "S&P"}
1152         ];
1153
1154         if(gameConfig.playable) {
1155             returnsAnnotations.push({"x": stock.traderReturn, "text":
... "You"});
1156         }
1157
1158         if(stock.competitor && stock.competitor.returns.length ===
... 1) {
1159             returnsAnnotations.push({"x": stock.competitor.returns[0],
... "text": "Challenger"});
1160         }
1161
1162         // console.log(stock.ticker + ': ' +
... JSON.stringify(result.returns.map(function(d) { return
... d.toPrecision(4); })));
1163
1164         if(window.innerWidth > 500) {
1165             var returnsHistogram = histogram()
1166                 .width(300)
1167                 .height(75)
1168                 .title('Returns')
1169                 .annotations(returnsAnnotations);
1170             svg.append("g.histogram.returns")
1171                 .attr("transform", "translate(" + (stock.stockReturn >
... -.2 ? 20 : (width-320)) + ",10)")
1172                 .datum(result.returns)
1173                 .call(returnsHistogram);
1174
1175             var tradeHistogram = histogram()
1176                 .x(x)
1177                 .height(100);
1178             svg.append("g.histogram.trades")
1179                 .attr("transform", "translate(0," + (height-100) + ")")
1180                 .datum([].concat.apply([],
... result.trades).map(f('date')))
1181                 .call(tradeHistogram);
1182         }
1183
1184         if(gameConfig.logging) {
1185             analysisStats.push({
1186                 "stock ticker": stock.ticker,
1187                 "start date": x.domain()[0],
1188                 "end date": x.domain()[1],
```

```
1189         "stock return": stock.stockReturn,
1190         "stock deviation": getStdevProper(stock.values),
1191         "returns mean": d3.mean(result.returns),
1192         "returns deviation": d3.deviation(result.returns),
1193         "trades number": result.trades.length,
1194         "trades duration": d3.mean(result.trades.map(function(d)
... { return (d[1].date - d[0].date) / (x.domain()[1] - x.domain()[0])
... })),
1195         "stock percentile": d3.bisect(result.returns,
... stock.stockReturn) / result.returns.length,
1196         "cash percentile": d3.bisect(result.returns, 0) /
... result.returns.length
1197     });
1198 }
1199
1200 }
1201
1202     if(gameConfig.autoplay) getNextLevel();
1203 }
1204
1205     return true;
1206 }
1207
1208     function gridlineTicks(d) {
1209         var gridTicks = [];
1210         var gridInterval = 10;
1211         for(var i = Math.ceil(y.domain()[0]/gridInterval); i <=
... Math.floor(y.domain()[1]/gridInterval); i++) {
1212             gridTicks.push(i*gridInterval);
1213         }
1214         return gridTicks;
1215     }
1216
1217     function addAlert(string, stick, tweet) {
1218         var alert = alerts.append("div.alert")
1219             .html(string);
1220
1221         if(!stick) {
1222             alert.transition()
1223                 .delay(4000)
1224                 .duration(500)
1225                 .remove();
1226         }
1227
1228         if(tweet) {
1229             alert.classed("tweetable", true)
1230                 .on("click", function() { postToTwitter(stock.challengeUrl,
... tweet); });
1231         }
1232
1233         alert.select("input").on("click", function() {
```

```
1234         this.setSelectionRange(0, this.value.length);
1235     })
1236 }
1237
1238     function showTip() {
1239         if(tips.length) addAlert(tips.shift());
1240     }
1241
1242 });
1243 }
1244
1245 /*  ——— GAME-OVER  ———
1246     Mostra riepilogo rendimenti + pulsante "Gioca ancora"
1247 */
1248 function gameOver () {
1249
1250     /*-- ri-mostra l'header --*/
1251     d3.select('#full-header').classed('hidden', false);
1252
1253     /*-- overlay centrale --*/
1254     const overlay = d3.select('.levels')
1255         .append('div')
1256         .attr('class', 'game-over')
1257         .style('z-index', d3.selectAll('.levels > div').size());    //
1258     ... sopra a tutto
1259
1260     const center = overlay.append('div').attr('class', 'center-middle');
1261
1262     /*—————
1263         1.  RIEPILOGO DEI RENDIMENTI
1264     —————*/
1265
1266     const fmt = d3.format('.1%');
1267
1268
1269
1270     const righe = titoliRend.map((r, i) =>
1271         ``Titolo ${i + 1} tuo rendimento: ${fmt(r)}``);    // ← ora r è un
1272     ... numero
1273
1274     `Titolo ${playedStockNames[i]} - tuo rendimento: ${fmt(r)}``);
1275     const mediaTitoli = d3.mean(titoliRend);    // funziona perché sono
1276     ... numeri
1277     const mediaSp500 = d3.mean(sp500Rend);
1278
1279     righe.push(`Media tuo rendimento titoli: ${fmt(mediaTitoli)}`);
1280     righe.push(`Media rendimento S&P500: ${fmt(mediaSp500)}`);
1281
```

```
1282  /*-- paragrafo riepilogo --*/
1283  center.append('div')
1284    .html(righe.join('<br>'))
1285    .style('font-size', '1.3em')
1286    .style('margin-bottom', '10px')
1287    .style('max-width', '90vw')
1288    .style('text-align', 'left');
1289
1290
1291  /* -----
1292  2. TITOLONE + PULSANTE "GIOCA ANCORA"
1293  -----*/
1294
1295  center.append('h2').text('GAME OVER');
1296
1297  center.append('button')
1298    .style('background', '#5bc0ff') // azzurro
1299    .style('color', '#00334d')      // testo blu scuro (contrasto)
1300    .text('Gioca ancora')
1301    // .on('click', () => location.reload()); // RIGA ORIGINALE
1302    .on('click', () => {              // <<< INIZIO MODIFICA
1303      cash = 500;                    // Resetta variabile
1304      localStorage.removeItem("cash"); // Rimuovi da localStorage
1305      // localStorage.setItem("cash", "500"); // Alternativa: Imposta
1306      // a 500
1307      location.reload();              // Ricarica pagina
1308    });                               // <<< FINE MODIFICA
1309
1310  /* -----
1311  3. CANVAS DECORATIVO (come versione originale)
1312  -----*/
1313  initEnderCanvas(overlay.append('canvas'));
1314
1315  // ... resto della funzione ...
1316
1317  /* -----
1318  3. CANVAS DECORATIVO (come versione originale)
1319  -----*/
1320  initEnderCanvas(overlay.append('canvas'));
1321
1322
1323  /* -----
1324  4. LOG DI DEBUG (se abilitato)
1325  -----*/
1326  if (gameConfig.logging) {
1327    console.table(analysisStats);
1328    console.table(cashRules);
1329  }
1330 }
1331
```

```
1332
1333 //////////////////////////////////////////////////
1334 // HELPERS AND STUFF
1335
1336 function getSmoothDomainFunction(data) {
1337
1338     // simplify price
1339     // tolerance comes from price
1340     var dataSimple = simplify(data.map(function(d) {
1341         return {
1342             "x": +d.date,
1343             "y": d.price
1344         }
1345     }), d3.mean(data.map(f('σ')))*1.5);
1346     var dataSimpleLine = dataSimple.map(function(d) {
1347         return {
1348             "date": new Date(d.x),
1349             "price": d.y
1350         }
1351     });
1352     // smooth price
1353     var smoothPath = Smooth(dataSimple.map(function(d) {
1354         return [d.x, d.y];
1355     }));
1356
1357     // simply variance
1358     // tolerance is just... high. too much volatility in aperture is
1359     ... disorienting.
1360     var dataStdevSimple = simplify(data.map(function(d) {
1361         return {
1362             "x": +d.date,
1363             "y": d.σ
1364         }
1365     }), 30);
1366     var dataStdevSimpleLine = dataStdevSimple.map(function(d) {
1367         return {
1368             "date": new Date(d.x),
1369             "price": d.y
1370         }
1371     });
1372     // smooth variance
1373     var smoothStdevPath = Smooth(dataStdevSimple.map(function(d) {
1374         return [d.x, d.y];
1375     }));
1376
1377     // example usage in situ: getSmoothDomain(tScale.invert(t));
1378     function getSmoothDomain(date) {
1379         // SMOOTH AND SIMPLIFY BOUNDS STUFF
1380         var bi = d3.bisect(dataSimpleLine.map(f('date')), date);
1381         if(bi < dataSimpleLine.length) {
1382             var biPc = (+date - +dataSimpleLine[bi-1].date) /
```

```
1381... (+dataSimpleLine[bi].date - +dataSimpleLine[bi-1].date)
1382     // console.log("We are " + Math.round(biPc*100) + "% between " +
... dataSimpleLine[bi-1].date + " and " + dataSimpleLine[bi].date);
1383     var smoothPrice = smoothPath((bi-1) + biPc)[1];
1384 } else {
1385     var smoothPrice = smoothPath(bi)[1];
1386 }
1387 // END SMOOTH AND SIMPLIFY BOUNDS STUFF
1388
1389 // SMOOTH AND SIMPLIFY VARIANCE STUFF
1390 var bi = d3.bisect(dataStdevSimpleLine.map(f('date')), date);
1391 if(bi < dataStdevSimpleLine.length) {
1392     var biPc = (+date - +dataStdevSimpleLine[bi-1].date) /
... (+dataStdevSimpleLine[bi].date - +dataStdevSimpleLine[bi-1].date)
1393     var smoothStdev = smoothStdevPath((bi-1) + biPc)[1];
1394 } else {
1395     var smoothStdev = smoothStdevPath(bi)[1];
1396 }
1397 // END SMOOTH AND SIMPLIFY VARIANCE STUFF
1398
1399 var aperture = smoothPrice * .2 + smoothStdev * 4; //original
1400 var aperture = smoothPrice * 0 + smoothStdev * 5; //for fed
... liftoff
1401
1402 var domain = [
1403     Math.max(smoothPrice - aperture, 0),
1404     smoothPrice + aperture
1405 ];
1406
1407 domain.price = smoothPrice;
1408 domain.stdev = smoothStdev;
1409
1410 return domain;
1411 }
1412
1413 getSmoothDomain.price = dataSimpleLine;
1414 getSmoothDomain.stdev = dataStdevSimpleLine;
1415
1416 return getSmoothDomain;
1417 }
1418
1419 function getStdev(d) {
1420     // compute from periodicity of data
1421     var period = Math.min(this[1].date - this[0].date, this[2].date -
... this[1].date);
1422     var varianceWindow = period * 90;
1423
1424     // N.B. #BUG:
1425     // IF PERIODICITY IS SUFFICIENTLY BIMODAL,
1426     // there can be gaps greater than the varianceWindow,
1427     // which leads to this returning 'undefined' stdev,
```

```
1428 // which makes it unreliable when fed into smooth+simplify
1429 // for stocks with huge gaps relative to the granularity
1430 // (e.g. a week of 5-minute data w/ nights and weekends)
1431
1432 var bounds = [
1433     new Date(+d.date - (.5 * varianceWindow)),
1434     new Date(+d.date + (.5 * varianceWindow))
1435 ];
1436 var boundedData = this.filter(function(dd) { return dd.date >
... bounds[0] && dd.date <= bounds[1]; });
1437
1438 d.σ = d3.deviation(boundedData, f('price'));
1439 }
1440
1441 // calculate stdev from intraday returns
1442 //
... http://www.investopedia.com/ask/answers/021015/how-can-you-calculate-
... volatility-excel.asp
1443 function getStdevProper(values) {
1444     values.forEach(function(d, i) {
1445         if(i === 0) return;
1446         d.return = (d.price / values[i-1].price) - 1;
1447     })
1448     return d3.deviation(values, f('return'));
1449 }
1450
1451 function getStdev2(d) {
1452     // compute from periodicity of data
1453     var period = Math.min(this[1].date - this[0].date, this[2].date -
... this[1].date);
1454     var varianceWindow = period * 90;
1455
1456     // N.B. #BUG:
1457     // IF PERIODICITY IS SUFFICIENTLY BIMODAL,
1458     // there can be gaps greater than the varianceWindow,
1459     // which leads to this returning 'undefined' stdev,
1460     // which makes it unreliable when fed into smooth+simplify
1461     // for stocks with huge gaps relative to the granularity
1462     // (e.g. a week of 5-minute data w/ nights and weekends)
1463
1464     var bounds = [
1465         new Date(+d.date - (.5 * varianceWindow)),
1466         new Date(+d.date + (.5 * varianceWindow))
1467     ];
1468     var boundedData = this.filter(function(dd) { return dd.date >
... bounds[0] && dd.date <= bounds[1]; });
1469
1470     boundedData.forEach(function(d, i) {
1471         if(i === 0) return;
1472         d.return = (d.price / boundedData[i-1].price) - 1;
1473     })
```

```
1474     d.σ2 = d3.deviation(boundedData, f('return'));
1475 }
1476
1477
1478 function timerFormat(t) {
1479     var secondsFormat = d3.format('02');
1480     return (Math.ceil(t/1000) == 60 ? "1" : "") + ":" +
1481     ... secondsFormat(Math.abs(Math.ceil(t/1000) % 60));
1482 }
1483
1484 // https://css-tricks.com/snippets/javascript/get-url-variables/
1485 function getQueryVariable(variable) {
1486     var query = window.location.search.substring(1);
1487     var vars = query.split("&");
1488     for (var i=0;i<vars.length;i++) {
1489         var pair = vars[i].split("=");
1490         if(pair[0] == variable){return pair[1];}
1491     }
1492     return(false);
1493 }
1494
1495 // CANVAS
1496
1497 function initCanvas() {
1498     var canvas = d3.select(".opener canvas");
1499     canvas.node().setAttribute('width', innerWidth);
1500     canvas.node().setAttribute('height', innerHeight);
1501     var ctx = canvas.node().getContext('2d');
1502
1503     initStocks();
1504
1505     function initStocks() {
1506         // n.b. hella clever random color
1507         ... http://www.paulirish.com/2009/random-hex-color-code-snippets/
1508         var stocks = d3.range(20).map(function(d) {
1509             return {
1510                 "t": 0,
1511                 "p": innerHeight * Math.random(),
1512                 "color": '#' + Math.floor(Math.random()*16777215).toString(16)
1513             };
1514         });
1515         d3.timer(function(t) { return renderCanvas(t, stocks); });
1516     }
1517
1518     function getPriceChange(stock) {
1519         var dp = (d3.random.logNormal(.5,1)() * -1 + 3) / 200;
1520         stock.p = stock.p + stock.p * dp;
1521     }
1522
1523     function renderCanvas(t, stocks) {
```

```
1523
1524 // if user has advanced, stop animation
1525 if(levelNumber > 0) return true;
1526
1527 // if time is up, restart timer
1528 if(t/20 > innerWidth) {
1529     initStocks();
1530     return true;
1531 }
1532
1533 stocks.forEach(function(d,i) {
1534
1535     ctx.strokeStyle = d.color;
1536
1537     if(d.p >= innerHeight) {
1538         d.p = innerHeight * Math.random();
1539         return;
1540     }
1541
1542     ctx.beginPath();
1543     ctx.moveTo(d.t/20, innerHeight - d.p);
1544
1545     getPriceChange(d);
1546     d.t = t % (innerWidth*20);
1547
1548     ctx.lineTo(d.t/20, innerHeight - d.p);
1549     ctx.stroke();
1550
1551 })
1552 }
1553 }
1554
1555 function initEnderCanvas(canvas) {
1556     var width = innerWidth;
1557     var height = innerHeight;
1558     canvas.node().setAttribute('width', width);
1559     canvas.node().setAttribute('height', height);
1560     var ctx = canvas.node().getContext('2d');
1561
1562     initStocks();
1563
1564     function initStocks() {
1565         // n.b. hella clever random color
1566         ... http://www.paulirish.com/2009/random-hex-color-code-snippets/
1567         var stocks = d3.range(20).map(function(d) {
1568             return {
1569                 "t": 0,
1570                 "p": Math.min(innerHeight, innerWidth) * Math.random() * .7,
1571                 "offset": Math.PI * 2 * Math.random(),
1572                 "color": '#' + Math.floor(Math.random()*16777215).toString(16)
1573             };
1574         });
1575     }
1576 }
```

```
1573     });
1574     d3.timer(function(t) { return renderCanvas(t, stocks); });
1575 }
1576
1577 function getPriceChange(stock) {
1578     var dp = (d3.random.logNormal(.5,1)()) * -1 + 3) / 200;
1579     stock.p = stock.p + stock.p * dp;
1580 }
1581
1582 function renderCanvas(t, stocks) {
1583
1584     var timeToAngle = d3.scale.linear()
1585         .domain([0,5000])
1586         .range([0,Math.PI*2]);
1587
1588     // if time is up, restart timer
1589     if(t/20 > innerWidth) {
1590         initStocks();
1591         return true;
1592     }
1593
1594     stocks.forEach(function(d,i) {
1595
1596         ctx.strokeStyle = d.color;
1597
1598         // get r and theta from d.p and t
1599         var angle = timeToAngle(d.t) + d.offset;
1600         var radius = d.p;
1601         var cartX = radius * Math.cos(angle) + width/2;
1602         var cartY = radius * Math.sin(angle) + height/2;
1603
1604         ctx.beginPath();
1605         ctx.moveTo(cartX, cartY);
1606
1607         getPriceChange(d);
1608         d.t = t;
1609
1610         var angle = timeToAngle(d.t) + d.offset;
1611         var radius = d.p;
1612         var cartX = radius * Math.cos(angle) + width/2;
1613         var cartY = radius * Math.sin(angle) + height/2;
1614
1615         ctx.lineTo(cartX, cartY);
1616         ctx.stroke();
1617
1618     })
1619 }
1620 }
1621
1622 //
1623 // ADS
```

```
1624
1625 function getBestAdSize(container) {
1626     /*
1627     IAB ad unit nomenclature:
1628     -----
1629     300 × 250    Medium Rectangle
1630     180 × 150    Rectangle
1631     160 × 600    Wide Skyscraper
1632     728 × 90     Leaderboard
1633     -----
1634     http://www.iab.net/guidelines/508676/508767/UAP
1635     */
1636     var sizes = [[300,250],[180,150],[160,600],[728,90]];
1637     var eligibleSizes = sizes.filter(function(d) { return d[0] <
1638     ... container.offsetWidth; });
1639     var widestSize = _.max(eligibleSizes,function(d) { return d[0]; });
1640     return widestSize;
1641 }
1642
1643 // SERVER
1644
1645 var lambda = false;
1646 if(typeof AWS !== 'undefined' && !isTerminal) {
1647     lambda = new AWS.Lambda({
1648         region: "us-east-1",
1649         accessKeyId: "AKIAJG4MLF2YGY5XX7UA",
1650         secretAccessKey: "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"
1651     });
1652 }
1653
1654 function saveTrades(stock, callback) {
1655
1656     if(!lambda) {
1657         if(callback) callback.call(this, new Error("Lambda not defined;
1658     ... AWS failed us!"));
1659         return false;
1660     }
1661
1662     var newItem = {
1663         "ticker": stock.ticker,
1664         "timestamp": +new Date(),
1665         "user": userId,
1666         "screen": [innerWidth, innerHeight],
1667         "competitor": stock.competitor ? stock.competitor.gameid : false,
1668         "attempts": userHistory[stock.ticker] ?
1669     ... userHistory[stock.ticker].attempts : false,
1670         "return": stock.traderReturn, // ".217"
1671         "cash": stock.cash, // [{date, cash}, {date, cash}]
1672         "trades": stock.trades // jsonification drops the .outstanding and
1673     ... .id array object props
1674 }
```

```
1671     }
1672
1673     var params = {
1674         "operation": "create",
1675         "Item": newItem
1676     }
1677
1678     lambda.invoke({
1679         FunctionName: gameConfig.saveServer,
1680         Payload: JSON.stringify(params)
1681     }, function(err, data) {
1682         if (!err) {
1683             // result is currently an empty object, but hey
1684             var result = JSON.parse(data.Payload)
1685
1686             // return a key to the thing just inserted
1687             var newKey = {
1688                 "ticker": newItem.ticker,
1689                 "timestamp": newItem.timestamp
1690             };
1691
1692             if(callback) callback.call(this, undefined, newKey);
1693         } else {
1694             console.log("err: " + err);
1695             if(callback) callback.call(this, err, undefined);
1696         }
1697     });
1698 }
1699
1700 }
1701
1702 function getTrades(ticker, timestamp, callback) {
1703     if(timestamp === undefined) {
1704         // get cached crowd trades, if available; fail over to AWS Lambda
1705         // Dynamo
1706         d3.json("data/leaderboards/" + ticker + ".json", function(error,
1707         data) {
1708             if(error || gameConfig.alwaysUseServer) {
1709                 getTradesServer(ticker, undefined, callback);
1710             } else {
1711                 data.trades.forEach(function(pair) {
1712                     pair.forEach(function(endpoint) {
1713                         endpoint.price = +endpoint.price;
1714                         endpoint.date = new Date(endpoint.date);
1715                     })
1716                 });
1717                 callback(error, data);
1718             }
1719         });
1720     } else {
1721         // can't use cached when we're looking for a specific timestamp
```

```
1720     getTradesServer(ticker, timestamp, callback);
1721   }
1722 }
1723
1724 function getTradesServer(ticker, timestamp, callback) {
1725   var params = {
1726     operation: "list",
1727     ticker: ticker,
1728     timestamp: timestamp,
1729     limit: gameConfig.readServerLimit
1730   }
1731
1732   paginateLambdaRequest([], params, function(err,data) {
1733
1734     if(gameConfig.logging) {
1735       cashRules.push(d3.sum(data.map(f('cash', '1', 'cash'))));
1736     }
1737
1738     var returnData = {
1739       "returns": data.map(f('return')),
1740       "trades": [].concat.apply([], data.map(f('trades')))
1741       // "nestedTrades": data.map(f('trades'))
1742     };
1743
1744     returnData.trades.forEach(function(pair) {
1745       pair.forEach(function(endpoint) {
1746         endpoint.price = +endpoint.price;
1747         endpoint.date = new Date(endpoint.date);
1748       })
1749     });
1750
1751     console.log("$" + ticker + ": " + returnData.trades.length + "
... trades from " + returnData.returns.length + " playthroughs.");
1752     // console.log("//// ALL DATA FOR " + ticker.toUpperCase() + "
... ////");
1753     // console.log(JSON.stringify(data));
1754     // console.log("//// COMPRESSED ////");
1755     // console.log(JSON.stringify(returnData));
1756     // console.log("//// END DATA FOR " + ticker.toUpperCase() + "
... ////");
1757
1758     callback(err, returnData);
1759   })
1760 }
1761
1762 function paginateLambdaRequest(results, params, callback) {
1763
1764   if(!lambda) {
1765     if(callback) callback.call(this, new Error("Lambda not defined;
... AWS failed us!"));
1766     return false;
```

```
1767     }
1768
1769     // console.log("requesting with", JSON.stringify(params));
1770
1771     lambda.invoke({
1772         FunctionName: gameConfig.readServer,
1773         Payload: JSON.stringify(params)
1774     }, function(err, data) {
1775
1776         if (!err) {
1777             data = JSON.parse(data.Payload)
1778             results = results.concat(data.Items);
1779
1780             console.log(results.length, data.Count, params.limit,
1781 ... data.LastEvaluatedKey);
1782
1783             if(data.LastEvaluatedKey !== undefined && (params.limit ===
1784 ... undefined || data.Count < params.limit)) {
1785                 if(params.limit) params.limit -= data.Count;
1786                 params.startKey = data.LastEvaluatedKey;
1787                 paginateLambdaRequest(results, params, callback);
1788             } else {
1789                 callback(err, results);
1790             }
1791
1792             } else {
1793                 console.log("err: " + err);
1794                 callback(err, results);
1795             }
1796         });
1797     }
1798
1799     function closest(accessor) {
1800         var bi = d3.bisector(accessor).right;
1801         return function(array, item) {
1802             var i = bi(array, item);
1803             var left = array[i-1];
1804             var right = array[i];
1805
1806             var dLeft = (left !== undefined) ? Math.abs(accessor(left) - item)
1807 ... : Infinity;
1808             var dRight = (right !== undefined) ? Math.abs(accessor(right) -
1809 ... item) : Infinity;
1810
1811             return dLeft <= dRight ? left : right;
1812         }
1813     }
1814
1815     function centerPopup(width, height) {
```

```
1814     var wLeft = window.screenLeft ? window.screenLeft : window.screenX;
1815     var wTop = window.screenTop ? window.screenTop : window.screenY;
1816     var left = wLeft + (window.innerWidth / 2) - (width / 2);
1817     var top = wTop + (window.innerHeight / 2) - (height / 2);
1818     return 'width=' + width + ',height=' + height + ',top=' + top +
...   ',left=' + left;
1819 }
1820
1821
1822
1823 //////////////////////////////////////////////////
1824 // STATS
1825
1826 // based on http://bl.ocks.org/mbostock/3048450
1827 function histogram() {
1828
1829     var width = 960,
1830         height = 500,
1831         x,
1832         title,
1833         annotations = [];
1834
1835     function chart(selection) {
1836         selection.each(function(values) {
1837
1838             if(!values || !values.length) return;
1839
1840             if(x === undefined) {
1841                 // Either fit scale to middle 98% of values...
1842                 values = values.filter(function(d) {
1843                     return d >= d3.quantile(values,.01) && d <
... d3.quantile(values,.99);
1844                 })
1845                 x = d3.scale.linear()
1846                     .domain(d3.extent(values))
1847                     .range([0, width]);
1848             } else {
1849                 // ...or fit values to scale.
1850                 values = values.filter(function(d) {
1851                     return d >= x.domain()[0] && d < x.domain()[1];
1852                 })
1853             }
1854
1855             // A formatter for counts.
1856             var formatCount = d3.format(",.0f");
1857
1858             // Use date scale for dates, percentage scale otherwise. Sloppy,
... yes.
1859             if(values[0] instanceof Date) {
1860                 var formatLabel = function(d) {
1861                     var extent = d3.extent(values)[1] - d3.extent(values)[0];
```

```
1862     var format = d3.time.format.multi([
1863         [".%L", function(d) { return extent < 1000; }],
1864         [":%S", function(d) { return extent < 1000 * 60; }],
1865         ["%I:%M", function(d) { return extent < 1000 * 60 * 60;
...     }],
1866         ["%I %p", function(d) { return extent < 1000 * 60 * 60 *
...     24; }],
1867         ["%a %d", function(d) { return extent < 1000 * 60 * 60 *
...     24 * 7; }],
1868         ["%b %d", function(d) { return extent < 1000 * 60 * 60 *
...     24 * 31; }],
1869         ["%B", function(d) { return extent < 1000 * 60 * 60 *
...     24 * 365; }],
1870         ["%Y", function() { return true; }]
1871     ]);
1872     return format(new Date(d));
1873 }
1874 } else {
1875     var formatLabel = d3.format(".0%");
1876 }
1877
1878 // Generate a histogram using twenty uniformly-spaced bins.
1879 var data = d3.layout.histogram()
1880     .bins(75)
1881     (values);
1882
1883 var y = d3.scale.linear()
1884     .domain([0, d3.max(data, function(d) { return d.y; })])
1885     .range([height, 0]);
1886
1887 var xAxis = d3.svg.axis()
1888     .scale(x)
1889     .orient("bottom")
1890     .tickFormat(formatLabel);
1891
1892 var svg = d3.select(this);
1893
1894 var titleText = svg.selectAll("text.title").data([title]);
1895 titleText.enter().append("text.title");
1896 titleText.text(f());
1897
1898 var bar = svg.selectAll(".bar")
1899     .data(data);
1900
1901 bar.exit().remove();
1902
1903 var barEnter = bar.enter().append("g")
1904     .attr("class", "bar");
1905
1906 svg.selectAll(".bar")
1907     .attr("transform", function(d) { return "translate(" +
```

```
1907... x(d.x) + "," + y(d.y) + "); });
1908
1909     barEnter.append("rect")
1910         .attr("x", 1);
1911
1912     svg.selectAll(".bar").select("rect")
1913         .attr("width", x(data[1].x) - x(data[0].x))
1914         .attr("height", function(d) { return height - y(d.y); });
1915
1916     // barEnter.append("text")
1917     //     .attr("dy", ".75em")
1918     //     .attr("y", 6)
1919     //     .attr("x", (x(data[1].x) - x(data[0].x)) / 2)
1920     //     .attr("text-anchor", "middle");
1921
1922     // svg.selectAll(".bar").select("text")
1923     //     .text(function(d) { return formatCount(d.y); });
1924
1925     svg.selectAll(".x.axis")
1926         .data([data])
1927         .enter().append("g")
1928         .attr("class", "x axis");
1929
1930     svg.selectAll(".x.axis")
1931         .attr("transform", "translate(0," + height + ")")
1932         .call(xAxis);
1933
1934     var annos = svg.selectAll("g.annotation")
1935         .data(annotations);
1936     var annoEnter = annos.enter().append("g.annotation");
1937     annoEnter.append("line");
1938     annoEnter.append("text");
1939     annos
1940         .attr("transform", function(d) { return "translate(" + x(d.x)
1941 + ",0)"; })
1942         .select("line")
1943         .attr("x1", 0)
1944         .attr("x2", 0)
1945         .attr("y1", function(d,i) { return .3*height + i * 12})
1946         .attr("y2", height);
1947     annos.select("text")
1948         .text(f('text'))
1949         .attr("y", function(d,i) { return .3*height + i * 12})
1950         .attr("dy", "-.5em");
1951     })
1952 }
1953
1954 chart.width = function(_) {
1955     if (!arguments.length) return width;
1956     width = _;
```

```
1957     return chart;
1958 };
1959
1960 chart.height = function(_) {
1961     if (!arguments.length) return height;
1962     height = _;
1963     return chart;
1964 };
1965
1966 chart.x = function(_) {
1967     if (!arguments.length) return x;
1968     x = _;
1969     return chart;
1970 };
1971
1972 chart.title = function(_) {
1973     if (!arguments.length) return title;
1974     title = _;
1975     return chart;
1976 };
1977
1978 chart.annotations = function(_) {
1979     if (!arguments.length) return annotations;
1980     _.sort(function(a,b) { return b.x - a.x; });
1981     annotations = _;
1982     return chart;
1983 };
1984
1985 return chart;
1986 }
1987
1988
```